

Claude Code Workshop — Facilitator Overview

What This Is

A hands-on, full-day workshop for non-technical professionals who want to use Claude Code — Anthropic's terminal-based AI coding assistant — to do real analytical and automation work. Participants interact primarily through the terminal CLI; no prior programming experience is assumed. The workshop requires paid Claude access (Pro, Max, or API billing) and is **a standalone professional workshop, entirely outside the IMB 2026 MADA student course's free-tools track.**

The One-Project Spine

The day is structured around a single business deliverable that grows in capability across all six blocks:

1. A retail CSV (`data/orders.csv`) is explored and analysed in plain English.
2. The analysis is packaged into a script and turned into a repeatable artifact.
3. Token consumption, model choice, and effort settings are examined and tuned.
4. Claude is connected to external tools (filesystem MCP, GitHub MCP) and opens a real pull request.
5. A second AI surface (Antigravity) is wired in via MCP for multi-tool use.
6. A recurring loop is set up so the analysis runs itself on a schedule.

By end of day every participant has shipped something — not just played with a chatbot.

Block Sequence

Block	Title	Focus
B1	Doorway & Setup	First <code>claude</code> command; login; orientation to the terminal as a conversation surface
B2	The Loop & The Goal	The Read-Plan-Edit-Verify loop; ask Claude to analyse the CSV; inspect what it does
B3	Turning the Dials / Reading the Meter	Model switching (<code>/model</code>), effort levels (<code>/effort</code>), <code>/usage</code> , prompt caching, the meter
B4	Giving Claude Hands (MCP + GitHub)	Add filesystem MCP; add GitHub MCP; Claude opens a pull request
B5	A Second Cockpit: Antigravity	Wire in Google Antigravity via MCP; multi-tool agent behaviour
B6	Make It Run Itself + Wrap	<code>/loop</code> for recurring runs; <code>/schedule</code> for persistent cloud schedules; reflection and wrap

Timings are the instructor's discretion. Suggested durations and natural break points live in `instructor-notes.md` — not here.

Prerequisites

Participants need before the day starts:

- A Claude account with **paid access** (Pro, Max subscription, or API billing enabled). Free-tier accounts will hit rate limits during hands-on blocks.
- **Node.js** (v18 or later) installed — required for MCP packages.
- A terminal that opens. On macOS: Terminal or iTerm. On Windows: PowerShell or Windows Terminal (Git Bash also works).
- Optional but useful: a GitHub account (needed for B4's pull-request step).

See `pre-work.md` for the participant-facing checklist.

Rendering the Slide Deck

```
quarto render slides/claude-code-workshop.qmd --to revealjs
```

This produces `slides/claude-code-workshop.html`. Open it in any browser. Quarto must be installed (`quarto --version` to check).

Opening the Notebook

```
jupyter lab notebook/token-consumption-lab.ipynb
```

The notebook ships pre-executed with sample data. To run it live against the API, set `ANTHROPIC_API_KEY` and execute:

```
jupyter nbconvert --to notebook --execute --inplace notebook/token-consumption-lab.ipynb
```

This costs a few cents (a dozen small API calls). Details in `instructor-notes.md`.

Cost Budget

See `instructor-notes.md` for per-participant cost estimates and the recommendation to use a Pro/Max subscription (flat-rate) rather than API billing for predictable workshop costs.

Verifying the Bundle

```
python claude-code-workshop/_build/verify_bundle.py
```

Prints `BUNDLE OK` if all assets are present and checks pass.